

数据集参考



TL;DR

Time Series Fault Location (TSFL) 是基于观测平台的指标数据对rjob的故障进行可疑节点定位的 DeepScan插件。其核心思想是异常节点与正常节点之间的行为差异（离群即异常），通过比较不同节点间的指标差异，从而找到故障节点。

DeepScan-TSFL每次排查会返回10个可疑节点并给出故障评分，该故障评分只体现相对关系。同时每个故障节点还会给出可疑指标用于进一步验证。

▼ Motivation

大规模GPU集群是“同步系统”，单点故障会导致训练任务失效，而“慢节点”则会拖慢训练速度。当故障发生时，日志数据可能产生缺失，这对后续的根本原因分析工作带来了更大的挑战。尽管Deepscan的现有功能已经能够处理大部分的日常故障，然而对于少部分故障的排查仍然需要投入大量人力。在排查过程中往往需要运维团队和算法团队各个领域的专家经验，有时甚至不得不重启所有机器才能解决问题。

事实上在排查过程中存在非常朴素的方法：找到和其他机器表现不同的机器，那台机器大概率就是故障节点。基于这一方法论，我们选用时序数据作为数据来源，通过比较不同机器在指标上的差异，找到故障节点。选用时序主要基于两个考量，其一是单一节点的个别指标缺失影响较小，其二是指标数据可以帮助排查Slow和Stop两类故障。

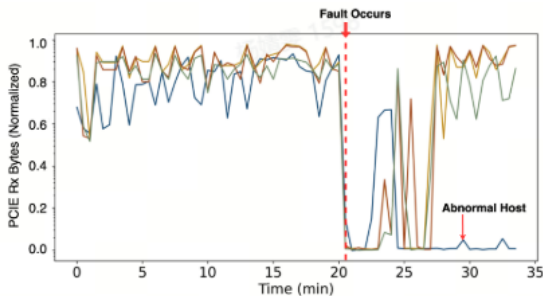


Figure 4. PCIe rx bytes pattern for each machine before and after a fault occurs.

故障机器的行为和正常机器有明显的差异

从时序角度出发，找到众多节点中的离群点可以视作是一种时间序列异常检测（TSAD）这一经典任务的变体，于是我们想通过TSAD方法解决故障定位的问题。

因此TSFL的目标是：

- 1. 减少专家经验的介入，能够识别之前从未发生过的故障
- 2. 降低对标签的需求（即尽可能做到无监督的故障定位）
- 3. 降低计算开销和维护负担，适应不断更新的运维系统

问题定义

根据我们在排查过程中的经验，故障定位等同于找到机器中的离群节点（离群即异常），因此我们将问题做如下定义：

假设我们有一个包含 N 台机器的集合，记为 M ,

$$M = \{m_1, m_2, \dots, m_N\}$$

为了衡量机器之间的不相似性，我们需要定义一个**不相似度函数**。设 $d(m_i, m_j)$ 为机器 m_i 和机器 m_j 之间的不相似度。这个函数衡量了任意两台机器在某些特征或行为上的差异程度。

对于集合 M 中的任意一台机器 m_k (其中 $k \in \{1, 2, \dots, N\}$)，我们可以计算它与其他所有机器的**总不相似度**，记为 $D(m_k)$ 。一种常见的计算方式是将其与集合中其他所有机器的不相似度进行累加：

故障节点定位问题可以被定义为：找到一台机器 m^* ，使得该机器的总不相似度最大。即，我们要找到的故障节点 m^* 满足：

$$m^* = \arg \max_{m_k \in M} D(m_k)$$

简而言之： 给定一个机器集合 M 和一个不相似度函数 $d(\cdot, \cdot)$ ，目标是找到机器 $m^* \in M$ ，该机器与其他所有机器的累积不相似度最高。

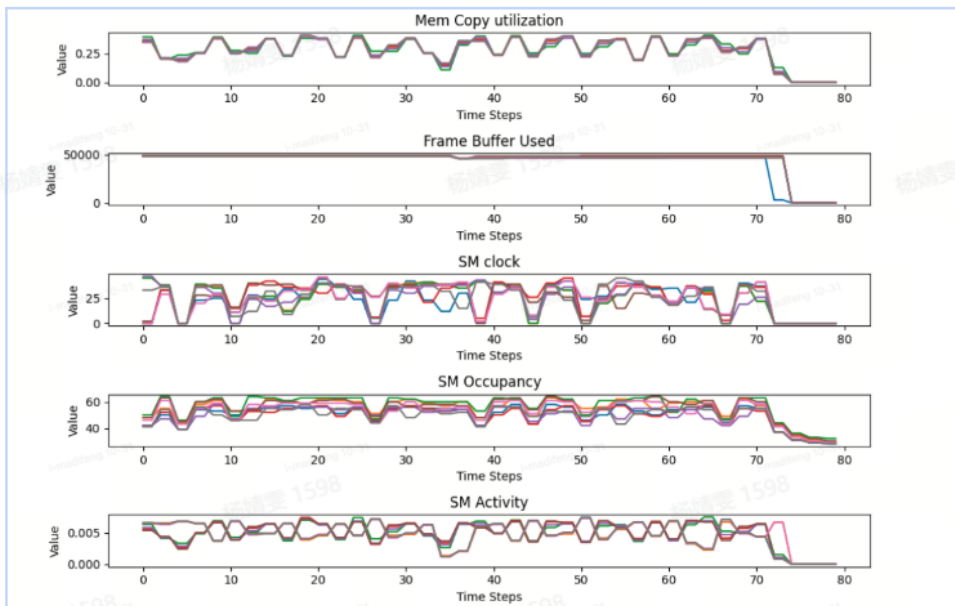
挑战

虽然我们尝试使用TSAD完成任务，然而直接将TSAD应用到GPU集群故障定位任务中仍然存在挑战：

- 高度动态且变化的正常模式
不同规模的任务的正常模式不同，当前的TSAD方法往往只能识别时序数据中的主要模式，一旦存在其他模式，都会被识别为异常

虽然我们尝试使用TSAD完成任务，然而直接将TSAD应用到GPU集群故障定位任务中仍然存在挑战：

- 高度动态且变化的正常模式
不同规模的任务的正常模式不同，当前的TSAD方法往往只能识别时序数据中的主要模式，一旦存在其他模式，都会被识别为异常
- 高维度且充满噪声的指标数据
一方面节点的状态需要由许多不同来源的指标反应，另一方面这些指标自身因为传感器抖动或采样软件Bug产生了大量的抖动。通过这些数据获得正常模式本身就十分困难



- 可扩展的统一分析框架
- 当前的TSAD方法可以根据输入的时序数量分为UTS（单指标）和MTS（多指标），尽管MTS看起来更加符合我们的场景，但是当输入指标数量增加时，MTS需要重新训练模型。

数据

我们使用的数据时间跨度从2024七月到2025一月。其中包含了98个故障的rjob（包含14个降速故障）和上百个无标签的rjob（我们也不知道该rjob是否正常退出），这些故障rjob的根因大多来自主线中的专家标注，具体的根因分布如下：

Table 2. Failure category and failure count of the cluster.

Category	Count	Likely Root Causes
GPU	51	Double bit error, non fatal XID error, unbalanced gpu utilization
Machine	13	Node down, machine check error
Network	18	Switch problem, optical module failure, cable failure, poor optical signal quality
PCIe	9	PCIe degradation, PCIe poor signal
NCCL	5	NCCL segmentation fault, NCCL Timeout
Others	2	Unknown

具体来说，我们使用了这些rjob中的如下时序数据，并将其分为了负载无关和负载相关的两类（带*的为负载无关），这种分类方法可以帮助

Table 1. Monitoring metrics used by TSLoc. Workload-independent metrics are marked with an asterisk.

Monitoring Metric	Description
CPU Usage	Percentage: Time the CPU is busy processing tasks.
Memory Usage	Percentage: System RAM currently used by programs and the OS.
RDMA Prio5 Throughput	Rate: Data transfer for high-priority Remote Direct Memory Access operations.
RDMA Prio0 Throughput	Rate: Data transfer for low-priority Remote Direct Memory Access operations.
GPU PCIe Throughput	Rate: Data transfer speed between GPU and motherboard via PCIe.
NVLink Throughput	Rate: Data transfer over the high-speed NVLink interconnect.
GPU Temp	Temperature: Current operating temperature of the Graphics Processing Unit.
Tensor Core Activity	Percentage: Utilization of GPU’s Tensor Cores for AI computations.
FP32 Activity	Percentage: Activity of single-precision floating-point units on GPU.
FP16 Activity	Percentage: Activity of half-precision floating-point units on GPU.
SM Activity	Percentage: Utilization of GPU’s Streaming Multiprocessors.
GPU Memory Usage	Percentage: GPU’s dedicated memory (VRAM) currently in use.
GPU Memory Copy Utility	Percentag: Efficiency or busyness of data transfers involving GPU memory.
GPU Usage	Percentage: Overall time the GPU is busy processing tasks.
PCIe Width*	Count: Number of active data lanes in the PCI Express connection.
Network Link Down Event*	Event: Occurrences of network interface connectivity loss.
Cable BER*	Rate: Bit errors during data transmission over a cable.
XID Errors*	Count: Occurrences of critical errors reported by the GPU.

通过收集参与rjob机器的以上所有时序指标，构造了训练集（无label，大约100个rjob）和测试集（98个故障rjob）。其中测试集的根本原因打在机器（Node）上。

方法

我们的方法想通过单变量的TSAD方法，无监督的学习时序的嵌入表示，并将嵌入表示作为衡量不同机器之间相似度的一个重要依据。

- 使用单变量时序（而不是多变量）：方便后续扩展到新的监控指标，而无需重新训练
- 使用TSAD模型：异常检测模型能够将原始时序有效压缩，保留其中的主要模式，在去噪的同时能压缩数据

设计

我们的方法分为离线和在线两个部分：

离线部分：

我们使用基于MoE的架构端到端的训练时间序列模型，具体的训练方法是让模型重构输入的时间序列窗口，Loss为MSE。通过这种方式，当模型能够很好的重构输入窗口时，我们认为模型学到时序中的主要模式。模型的具体架构是MoE，其中Gating网络是LSTM用于判断当前输入窗口应该被分配到下游的哪个专家，Expert网络则使用了[KAN-AD: Time Series Anomaly Detection with Kolmogorov-Arnold Networks](#)，该网络对于噪声的鲁棒能力较好，并且能够保持轻量化。

使用MoE的优点：

- 有良好的对抗灾难遗忘的能力，适合反复继续训练而不用担心出现性能崩溃
- Gating网络和Expert网络可以持续集成SOTA方法

当触发在线检测后，TSFL会爬取参与rjob的所有机器的相关指标。接下来TSFL会逐指标找到离群节点，最终根据机器在每个指标的离群分数排名进行聚合，最终得到故障分数的排名。

对于负载无关指标（比如PCIe Bandwidth）这种值不变的指标，跳过嵌入表示，直接通过和其他机器对比找到离群节点。进入这个分支得到的异常指标我们会直接赋10分（由于存在机器与机器之间比较和机器沿时间比较，最高会出现20分）

对于负载相关的指标，我们首先使用离线阶段获得的Encoder对原始时序进行嵌入（以滑动窗口的方式），得到一系列的嵌入表示后，逐时间步计算当前 node到其他node的距离和。对于该机器该指标的离群分数，通过加和所有时间步的到其他node的距离和得到。（对于缺失时间超过五分钟的指标，我们同样会在最终的聚合阶段增加其可疑分数）

聚合阶段，使用该机器在不同指标下的离群分数排名的倒数加和（排名第一为1分，第二为0.5分....）得到最终的故障分数。根据故障分数排名，我们可以得到故障分数排名前5的机器（Top@5）。除了给出最可能出现故障的节点外，我们还能给出对每个故障节点故障分数贡献最大的指标。

评估

我们使用Acc@5和Avg Rank对TSFL的准确度进行衡量，具体来说Acc@5表示根因机器出现在Top@5的rjob的比例，Avg Rank则是用于计算根因出现在Top@5的平均排名。前者衡量故障定位的准确率，后者用于衡量故障定位的效率。

Table 3. Comparison of Acc@5 and Avg Rank of solved cases between Mahalanobis Distance (MD) and Ours

	Acc@5	Avg Rank
MD	0.453	1.20
TSLoc	0.908	1.32

在98个Case上的评估结果

Table 5. Slow-down Failure Resolution Rate

	Acc@5	Avg Rank
Fail-Stop	0.952	1.32
Fail-Slow	0.643	1.33

在Slow Case上的评估结果